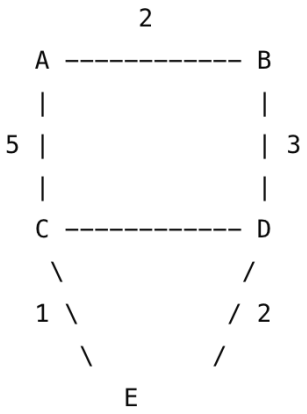
Exercise 2: Link State and Distance Vector Routing

Name: Yotsapat Panthong Student ID: 6710405478 Date: 21/9/2026

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2 a	5 a	Inf -	Inf -
1	b	2 a	5 a	5 b	Inf -
2	c	2 a	5 a	5 b	6 c
3	d	2 a	5 a	5 b	6 c
4	e	2 a	5 a	5 b	6c

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:

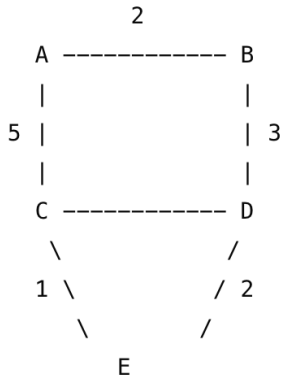
Destination	Shortest Path	Total Cost
B	a ->b	2
C	A c	5
D	A b c	5
E	A c e	6

Question 3 — Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	b	A b
C	5	c	A c
D	5	b	A b d
E	6	c	A c e

Part 2 — Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 — Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	2+0 = 2	5+4 = 9	2	b
C	2+4 = 6	5+0 = 5	5	c
D	2+3 = 5	5+1 = 6	5	b
E	2+5 = 7	5+1 = 6	6	c

Question 5 — Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	b	2	A b
C	c	5	A c
D	b	5	A b c
E	c	6	A c e

Part 3 — Routing Convergence

Question 6 — Link Failure

The C–D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.
Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

Distance Vector routing.
Your answer should describe:

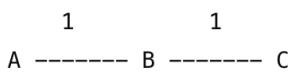
- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

Link State	Distance Vector
Router C and Router D detect the failure	Discover the failure locally via link-down detection or neighbor timeout. Other routers (A, B, and E) learn about the failure indirectly when they receive updated Distance Vector (DV) messages from their neighbors during the next scheduled update or triggered update.
Routers C and D generate new Link-State Advertisements (LSAs) indicating that the link between C and D is down	Router C updates its distance to D to INF. (or through E) and sends its updated vector to neighbors A and E.
Upon receiving the new LSA, each router validates the sequence number, replaces its old entry for the C–D link with the updated unreachability information, and synchronizes its local LSDB to match the rest of the network.	Distance Vector protocols lack global topology awareness. During propagation delay, a router might receive stale information from a neighbor that still advertises a path using the failed link before that neighbor learns of the failure (e.g., A might think C is still reachable through B, or vice versa). This can lead to transient routing loops or the count-to-infinity problem until split horizon, poison reverse, or hold-down timers stabilize the network.
Every router independently runs Dijkstra's algorithm to compute the new Shortest Path First (SPF) tree and routes	the network eventually converges after multiple rounds of DV exchanges until all routing tables stabilize with the correct costs.

*** in this part I use LLM to help finding answer***

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B–C link fails, but B receives stale information from A saying C is reachable at cost 2.

Question 7

1. Why can this create a routing loop?

Before router A or E learns the C–D link is broken, they may still advertise a path to D via C. Router C, seeing its direct link down, incorrectly accepts A's or E's stale advertisement and sends packets right back to them, trapping traffic in a cycle.

2. Show how the estimated distance to C can increase over successive iterations.

R1: B adds 1 to A's advertised cost (2+1 = 3) and sends this update to A. R2: A adds 1 to B's updated cost (3+1 = 4) and sends it back to B. R3: B adds 1 to A's cost (4+1 = 5). The cost will continue to increase back and forth over successive iterations.

3. What is count-to-infinity?

routing error where two or more routers continuously increment path costs back and forth in a loop, slowly counting up to a pre-defined maximum metric limit to finally mark the destination unreachable.

4. Why does it occur in Distance Vector routing?

It occurs because Distance Vector operates on "routing by rumor"—routers only know vector costs from immediate neighbors without knowing the full topology, rendering them unable to detect that a neighbor's advertised path relies on a link that has already failed.

5. Name a mitigation approach.

Split Horizon /with Poison Reverse

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Dijkstra	Bell-man
Information maintained	Full network topology database (LSDB)	Distance table of neighboring routers
Information exchanged	Link state of its own interfaces (LSPs) to everyone	owwn entire distance vector table to neighbors only
Algorithm	Global	Decentralized
Topology knowledge	Full	Partial
Routing updates	Event-driven only when topology change	Periodic intervals
Convergence	Fast	slow
Routing loops	Super rare	common
Count-to-infinity	Does not occur	Prone to occur
Example protocol	ospf	RIP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three

technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

ANS: For a 500-router enterprise network, *Link-State (LS) routing* is the ideal choice because it delivers near-instant convergence, complete topology visibility, and hierarchical scalability through features like OSPF Areas. While LS protocols have the disadvantages of using more router memory and CPU for running Dijkstra's algorithm, and can generate bandwidth spikes during network-wide flooding, these minor drawbacks are heavily outweighed by three major advantages: fast parallel path recalculation, guaranteed loop-free routing, and seamless multi-area partitioning that easily supports hundreds of devices without network instability.

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

Algorithmic Comparison: Link-State vs. Distance-Vector

- **Information Maintained:** Link-State routers maintain a complete topological map of every link in the network, resulting in a space complexity of $O(E)$. In contrast, Distance-Vector routers only track next-hop information for known destinations, keeping memory overhead lower at $O(N)$.
- **Communication Overhead:** LS protocols propagate topology changes network-wide via link-state advertisements (LSAs), generating an overhead of $O(N \times E)$. DV protocols exchange complete routing tables exclusively with directly connected neighbors, requiring $O(N)$ bandwidth per update cycle.
- **Computational Load:** LS relies on Dijkstra's algorithm to compute shortest paths locally, requiring $O(N \log N + E)$ processing time per node. DV uses the distributed Bellman-Ford algorithm, which distributes computation across neighbors and reduces per-device CPU load.
- **Convergence & Scalability:** LS achieves rapid convergence because each router independently calculates paths concurrently upon receiving topology updates. DV converges iteratively hop-by-hop, making it significantly slower and susceptible to transient issues like routing loops or the "count-to-infinity" problem.

While Link-State protocols demand more initial compute power and bandwidth, modern networking hardware handles these requirements easily. Furthermore, hierarchical designs (such as OSPF areas) effectively constrain LS flooding domains. Because routing loops and prolonged convergence times are unacceptable in large-scale enterprise environments, Link-State protocols provide the stability, speed, and scalability required for critical infrastructure.